

Impedance Learning for Robotic Contact Tasks Using Natural Actor-Critic Algorithm

Byungchan Kim, Jooyoung Park, Shinsuk Park, and Sungchul Kang

Abstract—Compared with their robotic counterparts, humans excel at various tasks by using their ability to adaptively modulate arm impedance parameters. This ability allows us to successfully perform contact tasks even in uncertain environments. This paper considers a learning strategy of motor skill for robotic contact tasks based on a human motor control theory and machine learning schemes. Our robot learning method employs impedance control based on the equilibrium point control theory and reinforcement learning to determine the impedance parameters for contact tasks. A recursive least-square filter-based episodic natural actor-critic algorithm is used to find the optimal impedance parameters. The effectiveness of the proposed method was tested through dynamic simulations of various contact tasks. The simulation results demonstrated that the proposed method optimizes the performance of the contact tasks in uncertain conditions of the environment.

Index Terms—Contact task, equilibrium point control, reinforcement learning, robot manipulation.

I. INTRODUCTION

OVER THE past decades, robotic technologies have remarkably advanced and have been proven to be successful, particularly in the field of manufacturing, where position-controlled robots perform simple repeated tasks in static environments. In the recent years, there have been increasing needs for robot systems in many areas that involve physical contacts with dynamic human-populated environments. Conventional robotic systems, however, have been ineffective in contact tasks.

Compared with robots, humans excel at contact tasks by skillfully coping with uncertain dynamic environments. They utilize their intrinsic compliance and force sensations during contact tasks. In addition, thanks to their excellent adaptation and learning abilities, various motor skills for various tasks are developed and stored in the central nervous system. When the

learned motor skill for a specific task is requested, the stored information is retrieved from the central nervous system.

The motor skill is also called the motor schema or the generalized motor program [1]. The motor schema describes the relationship between the outcomes achieved by past attempts and the parameters chosen in those attempts, and it is developed throughout the lifetime by practices and experiences [2]. The memorized motor schema could immediately be recalled for a particular movement. This feature enhances the movement efficiency and reduces the computational costs. In this sense, developing motor schemata through learning processes and exploiting these schemata for various tasks are promising methodologies for robotic tasks. In addition, in constructing motor schemata for robotic manipulation, it would be plausible to adopt the human strategy of managing contact tasks.

There have been several studies on biologically inspired motor learning. Cohen and Flash suggested the impedance learning strategy in a contact task by using associative search network [3]. They applied this approach to wall-following tasks. Another study on motor learning was performed by Izawa *et al.* [4]. They investigated a motor learning method for a musculoskeletal arm model in free-space motion using reinforcement learning. These studies, however, are limited to rather simple problems. Other studies employed artificial neural network models for impedance learning in contact tasks [5], [6]. One of the noticeable works of Tsuji *et al.* suggested an online virtual impedance learning method by exploiting visual information [6].

The purpose of this paper is to present a novel framework of motor skill learning for robotic contact tasks. To develop appropriate motor skills for various contact tasks, this paper employs the following methodologies. First, our robot control strategy employs impedance control based on a human motor control theory—the equilibrium point control model. The equilibrium point control model suggests that the central nervous system utilizes the *springlike* property of the neuromuscular system in coordinating multi-degrees of freedom human limb movements [7]. Under the equilibrium point control scheme, the force can separately be controlled by a series of equilibrium points and modulated stiffness (or more generally impedance) at the joints, so the control scheme can considerably become simplified. Second, as the learning framework, reinforcement learning is employed to optimize the performance of the contact task. Reinforcement learning can handle an optimization problem in an unknown environment by making sequential decision policies that maximize the external reward [8].

There have been several recent studies applying reinforcement learning methods to control problems (e.g., [10]–[14]).

Manuscript received February 13, 2009; revised May 22, 2009. First published August 18, 2009; current version published March 17, 2010. This work was supported by the Korean Institute of Construction and Transportation Technology Evaluation and Planning under Program 06-Unified and Advanced Construction Technology Program-D01. The work of J. Park was supported by the Ministry of Knowledge Economy under the Human Resources Development Program for Convergence Robot Specialists. This paper was recommended by Associate Editor A. Tayebi.

B. Kim and S. Kang are with the Center for Cognitive Robotics Research, Korea Institute of Science and Technology, Seoul 136-791, Korea (e-mail: bckim81@gmail.com; kasch804@gmail.com).

J. Park is with the Department of Control and Instrumentation Engineering, Korea University, Jochiwon 339-700, Korea (e-mail: parkj@korea.ac.kr).

S. Park is with the School of Mechanical Engineering, Korea University, Seoul 136-701, Korea (e-mail: drsspark@korea.ac.kr).

Digital Object Identifier 10.1109/TSMCB.2009.2026289

Particularly pertinent to this paper are studies that use natural actor-critic (NAC) methods [15]–[17], which show that they can quite effectively handle high-dimensional continuous state/action control problems. Park *et al.* [18] proposed a recursive least-square (RLS) filter combined with the NAC algorithm that can improve the computational efficiency of the original NAC [15].

This paper utilizes an episodic NAC (eNAC) algorithm based on the RLS filter for learning arm impedance during robotic contact tasks. A preliminary version of this paper, which reports on the application of the RLS-based NAC to learning task space stiffness, has appeared in [19]. Depending on the characteristics of the tasks, different task goals or performance indexes should be selected. In this paper, the performance indexes for three contact tasks (opening a door, point-to-point movement in an unknown force field, and catching a flying ball) were chosen to be optimized. The performance of each task was tested through dynamic simulations.

This paper is organized as follows: Section II introduces the equilibrium point control model of human motor control. In Section III, we describe the details of motor skill learning based on reinforcement learning. Finally, simulation results and discussion of the results are presented.

II. IMPEDANCE CONTROL BASED ON THE EQUILIBRIUM POINT CONTROL MODEL

The mechanical impedance of the human limb plays an important role in the dynamic interaction between the limb and its environment in contact [7], [20], [21]. In robotic applications, impedance control is a widely adopted method to execute robotic contact tasks by regulating its mechanical impedance, which characterizes the dynamic behavior of the robot at the point of interaction with its environment. The impedance control law can be described as [22]

$$\tau_{\text{actuator}} = \mathbf{J}^T(\mathbf{q}) (-\mathbf{K}_C(\mathbf{x} - \mathbf{x}_d) - \mathbf{B}_C\dot{\mathbf{x}}) + \mathbf{G}(\mathbf{q}) \quad (1)$$

where the vector τ_{actuator} represents the joint torques exerted by the actuators, and the vectors $\mathbf{x}$ and $\mathbf{x}_d$ are the current and desired positions of the end-effector, respectively. In the equation, matrices $\mathbf{K}_C$ and $\mathbf{B}_C$ represent the stiffness and damping matrices in Cartesian space. This form of impedance control is analogous to the equilibrium point control, which suggests that the joint torque generated by the muscles is proportional to the deviation of the instantaneous posture from its corresponding equilibrium posture. The equilibrium point control model proposes that the muscles and the neural control circuits have “springlike” properties, and the central nervous system may generate a series of equilibrium points for a limb. The “springlike” properties of the neuromuscular system will tend to drive the limb motion along a trajectory that follows these intermediate equilibrium postures [7], [20], [23]. Fig. 1 illustrates the concept of the equilibrium point control. Impedance control is an extension of the equilibrium point control in the context of robotics, where robot control is achieved by imposing the dynamic behavior described by the mechanical impedance at the end-effector.

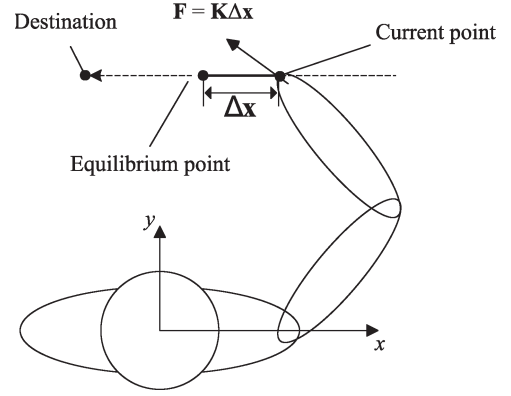


Fig. 1. Conceptual model of the equilibrium point control hypothesis.

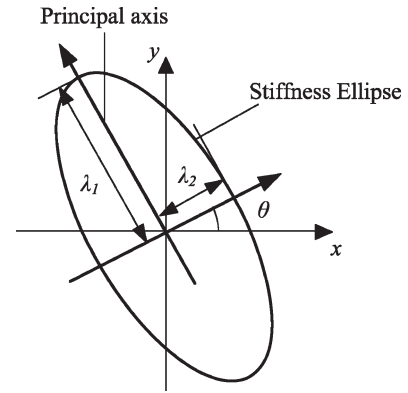


Fig. 2. Graphical representation of the end-effector's stiffness in Cartesian space. The lengths λ_1 and λ_2 of the principal axes and the relative angle θ represent the magnitude and the orientation of the end-effector's stiffness, respectively.

For impedance control of a planar manipulator, the stiffness matrix $\mathbf{K}_C$ is formed as

$$\mathbf{K}_C = \begin{bmatrix} K_{11} & K_{12} \\ K_{21} & K_{22} \end{bmatrix}. \quad (2)$$

The stiffness matrix $\mathbf{K}_C$ can be decomposed using singular value decomposition [24] as

$$\mathbf{K}_C = \mathbf{V}\mathbf{\Sigma}\mathbf{V}^T \quad (3)$$

where $\mathbf{\Sigma} = \begin{bmatrix} \lambda_1 & 0 \\ 0 & \lambda_2 \end{bmatrix}$, and $\mathbf{V} = \begin{bmatrix} \cos \theta & -\sin \theta \\ \sin \theta & \cos \theta \end{bmatrix}$.

In (3), the columns of the orthogonal matrix $\mathbf{V}$ are the eigenvectors of $\mathbf{K}_C$, and the diagonal elements of the diagonal matrix $\mathbf{\Sigma}$ are the eigenvalues of $\mathbf{K}_C$. The stiffness matrix $\mathbf{K}_C$ can graphically be represented by the stiffness ellipse, as shown in Fig. 2. In the figure, the eigenvectors and eigenvalues of $\mathbf{K}_C$ correspond to the directions and lengths of the principal axes of the stiffness ellipse, respectively [25]. The characteristics of the stiffness matrix $\mathbf{K}_C$ are determined by three parameters of its corresponding stiffness ellipse: 1) the magnitude (the area of the ellipse: $2\lambda_1\lambda_2$); 2) the shape (the length ratio of the major and minor axes: λ_1/λ_2); and 3) the orientation (the directions of the principal axes: θ). By regulating the three parameters, all the elements of the stiffness matrix $\mathbf{K}_C$ can be determined.

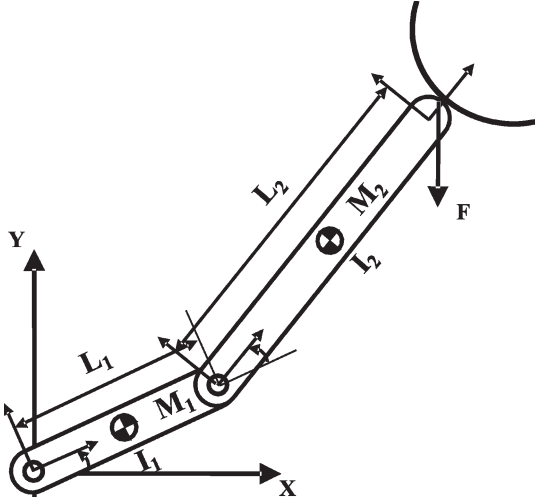


Fig. 3. Planar robotic manipulator (L_1 and L_2 : lengths of the links; M_1 and M_2 : masses of the links; and I_1 and I_2 : inertias of the links).

In this paper, the stiffness matrix in Cartesian space is assumed to be symmetric and positive definite as in planar human arm movement studies [7], [20]. This provides a sufficient condition for the static stability of the manipulator when it interacts with a passive environment [26]. It is also assumed that the damping matrix $\mathbf{B}_C$ is approximately proportional to the stiffness matrix $\mathbf{K}_C$. The ratio $\mathbf{B}_C/\mathbf{K}_C$ is chosen to be a constant of 0.05, as in the work of Won for human arm movement [27].

For trajectory planning, it is assumed that the trajectory of equilibrium point for the end-effector, which is also called the *virtual trajectory*, has a minimum-jerk velocity profile for the smooth movement of the arm [7], [20]. The virtual trajectory is calculated from the start point d_i to the final point d_f as

$$d(t) = d_i + (d_f - d_i) \left(10 \left(\frac{t}{t_f} \right)^3 - 15 \left(\frac{t}{t_f} \right)^4 + 6 \left(\frac{t}{t_f} \right)^5 \right) \quad (4)$$

where t is the current time, and t_f is the duration of movement.

III. REINFORCEMENT LEARNING OF MOTOR SKILL

This section describes how a robot manipulator learns the impedance modulation strategy for a specific task through reinforcement learning. A planar robotic manipulator for 2-D contact tasks was modeled as shown in Fig. 3. The robotic manipulator is controlled using the impedance control method based on the equilibrium point control hypothesis, as described in Section II. The stiffness and damping of the manipulator are modulated during a contact task, whereas the trajectory of equilibrium point is given for the task. Using the learning strategy based on reinforcement learning, the planar manipulator learns how to control the task impedance in performing 2-D contact tasks. In reinforcement learning, the state vector consists of the joint angles and velocities at the shoulder and elbow joints. The action vector changes the three parameters of the stiffness ellipse: 1) the magnitude (the area of the ellipse); 2) the shape (the length ratio of the major and minor axes);

and 3) the orientation (the direction of the principal axes). This section describes the learning method based on reinforcement learning for controlling the task impedance of the planar manipulator in performing 2-D contact tasks.

A. Reinforcement Learning

The main components of reinforcement learning are the decision maker, the agent, and the interaction with the external environment. General reinforcement learning problems can be represented via states $\mathbf{s} \in \mathcal{S}$, actions $\mathbf{a} \in \mathcal{A}$, rewards $r \in \mathbb{R}$, and time steps $t \in \{0, 1, 2, \dots\}$, in which the learning agent interacts with an environment [8]. At each time t , the agent observes the current state of the environment $\mathbf{s}_t$ and chooses an action $\mathbf{a}_t$ according to its policy in this state. It then receives a scalar reward r_t , which indicates the immediate desirability of the agent's action choice in state $\mathbf{s}_t$. The objective of the learning agent is to determine the policy π that, in some sense, optimizes the long-term performance in terms of received rewards. A commonly used long-term performance criterion is the expected sum of discounted rewards, which is defined as

$$J(\pi) \triangleq E \left\{ \sum_{k=0}^{\infty} \gamma^k r_k \middle| \mathbf{s}_0, \pi \right\} \quad (5)$$

where $\gamma \in (0, 1)$ is the discount rate, r_k is the immediate reward observed after the state transition from state $\mathbf{s}_k$ to state $\mathbf{s}_{k+1}$ (where $\mathbf{s}_0$ is the designated start state), and π denotes the policy from which actions are chosen. In general, the policy is described as the conditional probability $\pi(\mathbf{a}|\mathbf{s}) \triangleq \Pr\{\mathbf{a}_t = \mathbf{a} | \mathbf{s}_t = \mathbf{s}\}$. Under this criterion, the state value function and the action value function of a given policy are respectively defined as [8]

$$\begin{aligned} V^\pi(\mathbf{s}) &= E \left\{ \sum_{k=0}^{\infty} \gamma^k r_k \middle| \mathbf{s}_0 = \mathbf{s}, \pi \right\} \\ Q^\pi(\mathbf{s}, \mathbf{a}) &= E \left\{ \sum_{k=0}^{\infty} \gamma^k r_k \middle| \mathbf{s}_0 = \mathbf{s}, \mathbf{a}_0 = \mathbf{a}, \pi \right\}. \end{aligned} \quad (6)$$

With the so-called discounted state distribution $d^\pi(\mathbf{s}) \triangleq \sum_{k=0}^{\infty} \gamma^k \Pr\{\mathbf{s}_k = \mathbf{s} | \mathbf{s}_0, \pi\}$, one can rewrite the objective function (5) as [17], [28]

$$\begin{aligned} J(\pi) &= V^\pi(\mathbf{s}_0) = \int_{\mathcal{A}} \pi(\mathbf{a}|\mathbf{s}_0) Q^\pi(\mathbf{s}_0, \mathbf{a}) d\mathbf{a} \\ &= \int_{\mathcal{S}} d^\pi(\mathbf{s}) \int_{\mathcal{A}} \pi(\mathbf{a}|\mathbf{s}) r(\mathbf{s}, \mathbf{a}) d\mathbf{a} d\mathbf{s}. \end{aligned} \quad (7)$$

In the absence of a model of the environment dynamics and/or rewards, reinforcement learning algorithms can be used to determine an efficient policy π . In addition, the value functions associated with a policy can be computed, for example, by resorting to the TD(0) algorithm [8]. This algorithm uses the sampled transitions $(\mathbf{s}_t, \mathbf{a}_t, r_t, \mathbf{s}_{t+1})$ to update the estimate of the state value function as

$$V(\mathbf{s}_t) \leftarrow V(\mathbf{s}_t) + \alpha_t (r_t + \gamma V(\mathbf{s}_{t+1}) - V(\mathbf{s}_t)) \quad (8)$$

where α_t is the step-size sequence. This algorithm is known to generate a sequence convergent to a desired value function in tabular state spaces provided that the step-size sequence satisfies standard stochastic approximation conditions [29].

B. RLS-Based eNAC Algorithm

For many robotic problems, reinforcement learning schemes are required to deal with the continuous state/action space since learning methods based on discrete space are not applicable to high-dimensional systems. For solving continuous state/action system problems, the NAC algorithm [15]–[17] is known to be an effective approach. To improve the computational efficiency of the original NAC [15], Park *et al.* [18] suggested a modified NAC algorithm combined with an RLS filter. The RLS filter is used for adaptive signal filtering due to its fast convergence speed and low computational burden while the possibility of divergence is rather low [30]. Since the RLS filter in [18] was designed for continuing tasks with no final state (i.e., *non-episodic* tasks), its algorithm may not be efficient in dealing with *episodic* tasks. This paper considers a modified NAC algorithm combined with the RLS filter for *episodic* tasks. We name this algorithm the RLS-based eNAC algorithm. The RLS-based eNAC algorithm has two separate memory structures: 1) the *actor* structure and 2) the *critic* structure. The actor structure determines the policy that selects an action at each state, and the critic structure evaluates the selected action of the actor structure indicating whether the action is good or not.

The main role of the *actor* structure is to generate actions via a policy represented by a conditional probability. At each state $\mathbf{s} \in \mathcal{S}$, the action $\mathbf{a} \in \mathcal{A}$ is drawn in accordance with the conditional distribution $\pi_\psi(\mathbf{a}|\mathbf{s})$, where ψ is the parameter vector characterizing the distribution. Thus, the objective function in (7) we seek to maximize can be written as

$$J(\pi_\psi) = J(\psi) = \int_{\mathcal{S}} d^{\pi_\psi}(\mathbf{s}) \int_{\mathcal{A}} \pi_\psi(\mathbf{a}|\mathbf{s}) r(\mathbf{s}, \mathbf{a}) d\mathbf{a} d\mathbf{s}. \quad (9)$$

One of the convenient strategies for seeking the best distribution parameter ψ is to utilize the direction of $\nabla_\psi J(\psi)$, which is often called the policy gradient. Utilizing the famous policy gradient theorem [28], [31], along with the equality $\int_{\mathcal{A}} \nabla_\psi \pi(\mathbf{a}|\mathbf{s}) d\mathbf{a} = 0$ for $\forall \mathbf{s} \in \mathcal{S}$, the policy gradient can be written as

$$\begin{aligned} \nabla_\psi J(\psi) &= \int_{\mathcal{S}} d^{\pi_\psi}(\mathbf{s}) \int_{\mathcal{A}} \nabla_\psi \pi_\psi(\mathbf{a}|\mathbf{s}) Q^{\pi_\psi}(\mathbf{s}, \mathbf{a}) d\mathbf{a} d\mathbf{s} \\ &= \int_{\mathcal{S}} d^{\pi_\psi}(\mathbf{s}) \int_{\mathcal{A}} \nabla_\psi \pi_\psi(\mathbf{a}|\mathbf{s}) (Q^{\pi_\psi}(\mathbf{s}, \mathbf{a}) - V^{\pi_\psi}(\mathbf{s})) d\mathbf{a} d\mathbf{s} \\ &= \int_{\mathcal{S}} d^{\pi_\psi}(\mathbf{s}) \int_{\mathcal{A}} \pi_\psi(\mathbf{a}|\mathbf{s}) \nabla_\psi \log \pi_\psi(\mathbf{a}|\mathbf{s}) A^{\pi_\psi}(\mathbf{s}, \mathbf{a}) d\mathbf{a} d\mathbf{s} \end{aligned} \quad (10)$$

where $A^{\pi_\psi}(\mathbf{s}, \mathbf{a}) \triangleq Q^{\pi_\psi}(\mathbf{s}, \mathbf{a}) - V^{\pi_\psi}(\mathbf{s})$ is the advantage value function [32], which gives the advantage of action $\mathbf{a}$ over the average value in state $\mathbf{s}$. According to an observation intro-

duced in [28] and [31], the advantage value function $A^{\pi_\psi}(\mathbf{s}, \mathbf{a})$ can be replaced by the so-called compatible approximator¹

$$A_w(\mathbf{s}, \mathbf{a}) \triangleq \nabla_\psi \log \pi_\psi(\mathbf{a}|\mathbf{s})^T \mathbf{w} \quad (11)$$

without affecting the unbiasedness of the gradient estimate. Note that the compatible approximator $A_w(\mathbf{s}, \mathbf{a})$ is linear with respect to the parameter vector $\mathbf{w}$. Based on (10) and (11), one can see that a desirable function form for an estimate of the policy gradient can be given as

$$\begin{aligned} \nabla_\psi J(\psi) &= \int_{\mathcal{S}} d^{\pi_\psi}(\mathbf{s}) \int_{\mathcal{A}} \pi_\psi(\mathbf{a}|\mathbf{s}) \nabla_\psi \log \pi_\psi(\mathbf{a}|\mathbf{s}) A^{\pi_\psi}(\mathbf{s}, \mathbf{a}) d\mathbf{a} d\mathbf{s} \\ &\approx \int_{\mathcal{S}} d^{\pi_\psi}(\mathbf{s}) \int_{\mathcal{A}} \pi_\psi(\mathbf{a}|\mathbf{s}) \nabla_\psi \log \pi_\psi(\mathbf{a}|\mathbf{s}) A_w(\mathbf{s}, \mathbf{a}) d\mathbf{a} d\mathbf{s} \\ &= \mathbf{F}(\psi) \mathbf{w} \end{aligned} \quad (12)$$

where

$$\begin{aligned} \mathbf{F}(\psi) &\triangleq \int_{\mathcal{S}} d^{\pi_\psi}(\mathbf{s}) \int_{\mathcal{A}} \pi_\psi(\mathbf{a}|\mathbf{s}) \\ &\quad \times \nabla_\psi \log \pi_\psi(\mathbf{a}|\mathbf{s}) \nabla_\psi \log \pi_\psi(\mathbf{a}|\mathbf{s})^T d\mathbf{a} d\mathbf{s}. \end{aligned} \quad (13)$$

Among a variety of powerful gradient-based algorithms, one of the most efficient tools for updating the parameter vector ψ of $\pi_\psi(\mathbf{a}|\mathbf{s})$ would be the natural gradient method [33]. This method does not follow the steepest direction in the parameter space but the steepest direction with respect to the Fisher metric given by $\tilde{\nabla}_\psi J(\psi) = \mathbf{G}^{-1}(\psi) \nabla_\psi J(\psi)$, where $\mathbf{G}(\psi)$ denotes the Fisher information metric. In a recent remarkable paper of Peters *et al.* [15], it was shown that, for the reinforcement learning problem, the matrix $\mathbf{F}(\psi)$ is exactly the same with the Fisher information matrix; thus, the natural gradient of $J(\psi)$ can be estimated using

$$\tilde{\nabla}_\psi J(\psi) = \mathbf{G}^{-1}(\psi) \nabla_\psi J(\psi) \approx \mathbf{G}^{-1}(\psi) \mathbf{F}(\psi) \mathbf{w} = \mathbf{w}. \quad (14)$$

This result enables us to update the actor parameter ψ via the following simple rule:

$$\psi \leftarrow \psi + \alpha \tilde{\nabla}_\psi J(\psi) \approx \psi + \alpha \mathbf{w} \quad (15)$$

where $\alpha > 0$ is the learning rate. Note that the actor structure described above is that of the NAC algorithm in [15]–[17].

In the *critic* structure, the least-square temporal-difference Q algorithm can be used for minimizing the TD error, which represents the deviation between the expected return and the current prediction value [34]. From the Bellman equation [15] [17], the action value function $Q^\pi(\mathbf{s}, \mathbf{a})$ satisfies

$$\begin{aligned} Q^\pi(\mathbf{s}, \mathbf{a}) &= A^\pi(\mathbf{s}, \mathbf{a}) + V^\pi(\mathbf{s}) \\ &= r(\mathbf{s}, \mathbf{a}) + \gamma \int_{\mathcal{S}} p(\mathbf{s}'|\mathbf{s}, \mathbf{a}) V^\pi(\mathbf{s}') d\mathbf{s}'. \end{aligned} \quad (16)$$

¹In this paper, we assume that the policy distribution π_ψ is such that the gradient $\nabla_\psi \log \pi_\psi(\mathbf{a}|\mathbf{s})^t$ is well defined.

This shows that, through the sampled transitions (s_t, a_t, r_t, s_{t+1}) , the left-hand side term $Q^\pi(s_t, a_t)$ can be approximated by $r_t + \gamma V^\pi(s_{t+1})$. Thus, $r_t + \gamma V^\pi(s_{t+1}) - V^\pi(s_t)$ becomes a valid approximation for the advantage function value $A^\pi(s_t, a_t)$. Adding these advantage values with proper discounts across N steps (which can cover a whole episode) yields the following approximation:

$$\sum_{t=0}^N \gamma^t A^\pi(s_t, a_t) \approx \sum_{t=0}^N \gamma^t r_t - V^\pi(s_0) + \gamma^{N+1} V^\pi(s_{N+1}). \quad (17)$$

The last term of the equality disappears for $N \rightarrow \infty$ or for episodic tasks terminating before N steps. Thus, by inserting the compatible approximator [see (11)] for the advantage function together with using a single variable v for the value of the start state $V^\pi(s_0)$, we get the following regression problem [15]:

$$\sum_{t=0}^N \gamma^t [\nabla_{\psi} \log \pi(a_t | s_t)^T, 1] \begin{bmatrix} \mathbf{w} \\ v \end{bmatrix} \approx \sum_{t=0}^N \gamma^t r_t. \quad (18)$$

Note that, here, the filter coefficients $\mathbf{w}$ and v are to be found to minimize the least-square error. With $\tilde{\phi}_i \triangleq \sum_{t=0}^N \gamma^t [\nabla_{\psi} \log \pi(a_t | s_t)^T, 1]^t$ and $\tilde{r}_i \triangleq \sum_{t=0}^N \gamma^t r_t$, which are computed using the scenario of the i th episode, the regression after the e th episode can compactly be written as

$$\begin{bmatrix} \tilde{\phi}_1^T \\ \vdots \\ \tilde{\phi}_e^T \end{bmatrix} \chi_e \approx \begin{bmatrix} \tilde{r}_1 \\ \vdots \\ \tilde{r}_e \end{bmatrix} \quad (19)$$

where $\chi_e \triangleq [\mathbf{w}_e^T, v_e]^T$.

From the standard least-square method [30], the filter coefficients minimizing the least-square error must satisfy the normal equation:

$$\mathbf{A}_e^T \mathbf{A}_e \chi_e = \mathbf{A}_e^T \mathbf{d}_e, \quad i.e., \quad \left(\sum_{i=1}^e \tilde{\phi}_i \tilde{\phi}_i^T \right) \chi_e = \sum_{i=1}^e \tilde{\phi}_i \tilde{r}_i \quad (20)$$

where $\mathbf{A}_e$ is the data matrix defined as $\mathbf{A}_e \triangleq [\tilde{\phi}_1, \dots, \tilde{\phi}_e]^T$, and $\mathbf{d}_e \triangleq [\tilde{r}_1 \dots \tilde{r}_e]^T$ is the vector of desired signals. When there is a need to put more emphasis on recent episodes, the use of the so-called forgetting factor $\beta \in (0, 1)$ is desirable [35]. In this case, the following normal equation needs to be used instead of (20):

$$\mathbf{M}_e \chi_e = \mathbf{b}_e \quad (21)$$

where $\mathbf{M}_e \triangleq \sum_{i=1}^e \beta^{e-i} \tilde{\phi}_i \tilde{\phi}_i^T$, and $\mathbf{b}_e \triangleq \sum_{i=1}^e \beta^{e-i} \tilde{\phi}_i \tilde{r}_i$. Note that for $e > 1$, $\mathbf{M}_e$ and $\mathbf{b}_e$ can be written in the following recursive form:

$$\begin{aligned} \mathbf{M}_e &= \beta \mathbf{M}_{e-1} + \tilde{\phi}_e \tilde{\phi}_e^T \\ \mathbf{b}_e &= \beta \mathbf{b}_{e-1} + \tilde{\phi}_e \tilde{r}_e. \end{aligned} \quad (22)$$

TABLE I
RLS-BASED eNAC ALGORITHM

Initialize each parameter vector: $\psi = \psi_0, \chi = \chi_0, \mathbf{M} = \mathbf{0}, \mathbf{P} = \mathbf{0}$
For each episode,
Run simulator:
For each step,
Take action $\mathbf{a}_t$ from stochastic policy π , then observe next state s_{t+1} and reward r_t
end
Update Critic structure:
if first update,
Update critic information matrices, following the initial update rule in (23).
else,
Update critic information matrices, following the recursive least-squares update rule in (24).
end,
Update Actor structure:
Update policy parameter vector, following the rule in (15).
repeat until termination

In addition, note that when $\mathbf{M}_e$ is invertible, the optimal solution to the normal equation [see (21)] is obviously $\chi_e = \mathbf{M}_e^{-1} \mathbf{b}_e$. However, $\mathbf{M}_e$ is usually not invertible until a sufficient number of episodes have been included in its summation. A common strategy used in the RLS method for ensuring the invertibility of $\mathbf{M}_e$ is to use $\delta \mathbf{I}$ for its initialization [30]. Employing the strategy, we have the following rules after the first episode:

$$\begin{aligned} \mathbf{M}_1 &\leftarrow \mathbf{M}_1 + \delta \mathbf{I} \\ \mathbf{P}_1 &= \mathbf{M}_1^{-1} \\ \chi_1 &= \mathbf{P}_1 \mathbf{b}_1. \end{aligned} \quad (23)$$

In (23), δ is a positive scalar constant, and $\mathbf{I}$ is the identity matrix. By the addition of the term $\delta \mathbf{I}$ at the first row of (23), matrix $\mathbf{M}_1$ becomes nonsingular [30]. As the update progresses, the effect of perturbation is diminished. For updates after the initialization, the matrix inversion formula [30] is applied to (22) for $\mathbf{M}_e$, which leads to the following recursive update rule for χ_e and $\mathbf{P}_e$:

$$\begin{aligned} \mathbf{k}_e &= \frac{\mathbf{P}_{e-1} \tilde{\phi}_e}{\beta + \tilde{\phi}_e^T \mathbf{P}_{e-1} \tilde{\phi}_e} \\ \chi_e &= \chi_{e-1} + \mathbf{k}_e (\tilde{r}_e - \tilde{\phi}_e^T \chi_{e-1}) \\ \mathbf{P}_e &= \frac{1}{\beta} \left(\mathbf{P}_{e-1} - \frac{\mathbf{P}_{e-1} \tilde{\phi}_e \tilde{\phi}_e^T \mathbf{P}_{e-1}}{\beta + \tilde{\phi}_e^T \mathbf{P}_{e-1} \tilde{\phi}_e} \right). \end{aligned} \quad (24)$$

Note that the resultant RLS solution $\chi_e = [\mathbf{w}_e^T, v_e]^T$ will be used in the update process for the actor part via (15). The entire procedure of the RLS-based eNAC algorithm is summarized in Table I.

C. Stochastic Action Selection

As discussed in Section II, the characteristics of the stiffness ellipse can be changed by modulating three parameters:

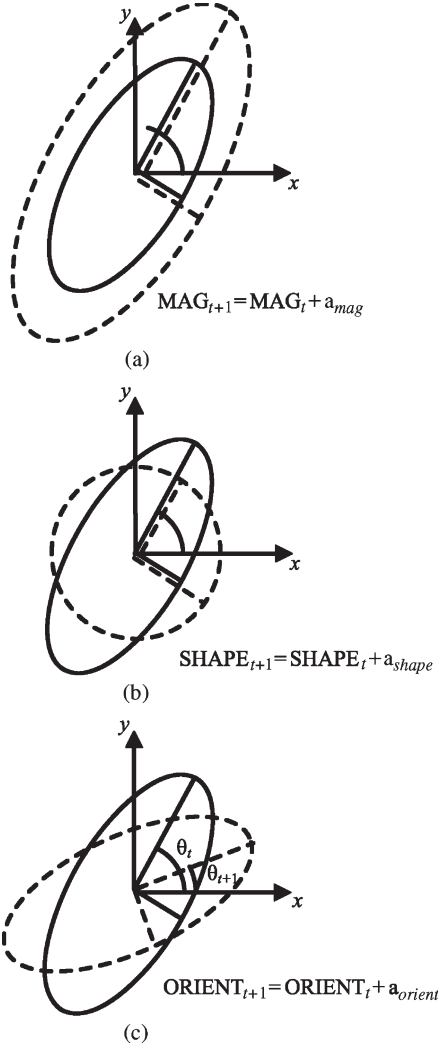


Fig. 4. Change of stiffness ellipse for one time step. (a) The magnitude (area of the ellipse). (b) The shape (length ratio of the major and minor axes). (c) The orientation (direction of the major axis). (Solid line: stiffness ellipse at time t ; dashed line: stiffness ellipse at time $t + 1$.)

1) magnitude; 2) shape; and 3) orientation. For performing contact tasks, policy π is designed to plan the change rate of the magnitude, shape, and orientation of the stiffness ellipse at each state by taking actions ($\mathbf{a}_t = [a_{\text{mag}}, a_{\text{shape}}, a_{\text{orient}}]^T$). Through the sequence of an episode, policy π determines those actions. The goal of the learning algorithm is to find the trajectory of the optimal stiffness ellipse during the episode. Fig. 4 illustrates the change of stiffness ellipse corresponding to each action.

Policy π is in the form of a Gaussian density function. In the critic structure, a compatible function approximation $\nabla_{\psi} \log \pi(\mathbf{a}_t | \mathbf{s}_t)^T \mathbf{w}$ is derived from the stochastic policy π based on the algorithm suggested by Williams [36]. Policy π for each component of action vector $\mathbf{a}_t$ can be described as follows:

$$\pi(a_t | \mathbf{s}_t) = N(a_t | \mu, \sigma) = \frac{1}{\sigma \sqrt{2\pi}} \exp\left(-\frac{(a_t - \mu)^2}{2\sigma^2}\right). \quad (25)$$

Since the stochastic action selection is dependent on the conditions of the Gaussian density function, the action policy

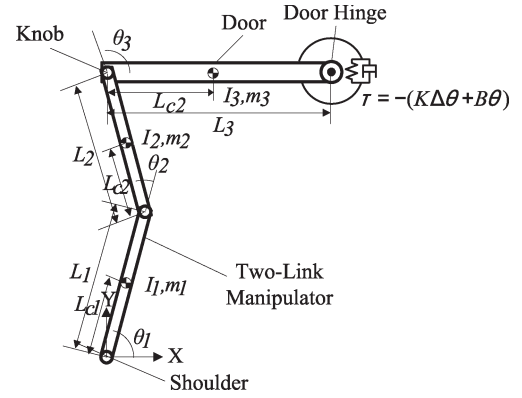


Fig. 5. Door-opening task. The end-effector is connected with the doorknob by a revolute joint hinge. The door hinge has rotational spring and damper.

can be designed by controlling the mean μ and variance σ of (25). These variables are defined as

$$\begin{aligned} \mu &= \xi \psi_*^T \mathbf{s}_t, \\ \sigma &= \tilde{\xi} \left(0.001 + \frac{1}{1 + \exp(-\eta)} \right). \end{aligned} \quad (26)$$

As can be seen in the equation, the mean μ is a linear combination of the vector ψ_* and the state vector $\mathbf{s}_t$ with the mean scaling factor ξ . The variance σ is in the form of the sigmoid function with positive scalar parameter η and variance scaling factor $\tilde{\xi}$. As a planar manipulator model is used in this paper, the components of the state vector $\mathbf{s}_t$ include the joint angles and velocities of the shoulder and elbow joints ($\mathbf{s}_t = [x_1, x_2, \dot{x}_1, \dot{x}_2, 1]^T$, where the fifth component of 1 is a bias factor). Therefore, the policy parameter vector ψ consists of 18 components, as follows:

$$\psi = [\psi_{\text{mag}}^T, \psi_{\text{shape}}^T, \psi_{\text{orient}}^T, \eta_{\text{mag}}, \eta_{\text{shape}}, \eta_{\text{orient}}]^T \quad (27)$$

where ψ_{mag} , ψ_{shape} , and ψ_{orient} are each a five-by-one vector, and η_{mag} , η_{shape} , and η_{orient} are each a scalar, as shown in (26).

IV. CONTACT TASK APPLICATIONS

In this section, the method developed in the previous section is applied to three contact tasks: 1) opening a door with an unknown length; 2) point-to-point movement in an unknown force field; and 3) catching a flying ball. The planar manipulator developed in Section III was used to perform the tasks in 2-D space. The dynamic simulator is constructed by MSC.ADAMS2005, and the control algorithm is implemented using Matlab/Simulink (Mathworks, Inc.).

A. Opening a Door With an Unknown Length

Fig. 5 illustrates the planar robotic manipulator opening a door in the horizontal plane. The physical properties of the arm are listed in Table II. The length of the door is unknown. Since the dimension of the door is uncertain, the robot may experience unexpected task failures through exerting excessive contact force on the doorknob. The door hinge has rotational spring and damper with unknown coefficients. To successfully

TABLE II
PHYSICAL PROPERTIES OF THE DOOR-OPENING TASK MODEL

	Length(m)	Mass(Kg)	Inertia(Kg · m ²)
Link 1	0.40	8.0	0.64
Link 2	0.40	3.7	0.30
Door	0.60	10.0	1.80

perform the door-opening task under those uncertainties, the following conditions should be satisfied:

- 1) The contact force normal to the trajectory of the doorknob should be maintained under a certain limit value so that the robot does not exert excessive force on the doorknob.
- 2) The door should be opened to a certain angle within a time limit.

Based on the conditions, two performance indexes are chosen, as follows:

- 1) the root mean square of the difference between the desired position (virtual trajectory) and the actual position of the endpoint ($\Delta_{\text{rms}} = \sqrt{(x_d - x)^2 + (y_d - y)^2}$);
- 2) the magnitude of contact force normal to the trajectory of the doorknob ($\|\mathbf{F}_{\text{normal}}\| = \sqrt{F_x^2 + F_y^2}$).

By minimizing Δ_{rms} , the movement can be smoothed, and the error between the desired goal position and the actual arrival position can be reduced. By minimizing $\|\mathbf{F}_{\text{normal}}\|$, the stability and safety of the task can be improved. Using the two performance indexes, the reward for one episode can be formulated as

$$\text{reward} = \nu_1 \left(\kappa_1 - \sum_{t=1}^N (\Delta_{\text{rms}})_t \right) + \nu_2 \left(\kappa_2 - \sum_{t=1}^N \|\mathbf{F}_{\text{normal}}\|_t \right) \quad (28)$$

where ν_1 and ν_2 are the weighting factors, and κ_1 and κ_2 are constants. Now, the problem has become the maximization of the reward through learning. The values of the weight factors were determined by considering the convergence during the learning phase and the balance of the units of force and displacement. The learning parameters were chosen as follows: $\alpha = 0.005$, $\beta = 0.99$, and $\gamma = 0.99$. From the initial state, the stiffness ellipse was modulated using the action policy. The change limits for action are set as $[-10, 10]$ degrees and $[-1, 1]$ for the orientation and the major/minor ratio of the stiffness ellipse, respectively. The initial stiffness ellipse was set to have 0° orientation and 4:1 ratio. It was assumed that the magnitude of the stiffness ellipse is fixed constant (10000π) during the task.

Dynamic simulations were carried out for three different virtual trajectories (understeering, oversteering, and exact steering), as shown in Fig. 6. The uncertainty in the length of the door, and thus the doorknob position, is modeled using three virtual trajectories. Fig. 7 illustrates the change of stiffness ellipses during the door-opening task for the three different virtual trajectories. The stiffness ellipses change as its shape

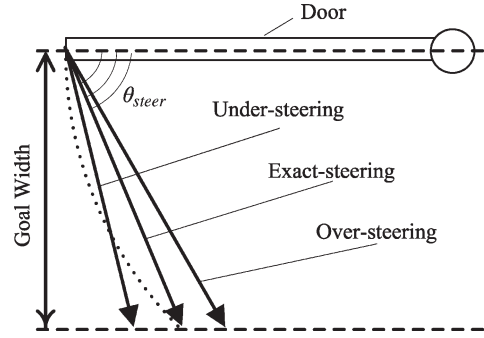


Fig. 6. Virtual trajectories for door opening (dotted line: actual trajectory of doorknob).

and orientation are tuned by the action determined by the policy. The figure shows the results after 200 learning iterations. As can be seen in the figure, the orientation and shape of the stiffness ellipses are similar for the three virtual trajectories. The major axis of the stiffness ellipse is nearly tangential to the trajectory of the doorknob, and the stiffness ellipse has a rather sharp shape. These tendencies can be interpreted that the stiffness tangential to the door knob trajectory was increased to reduce the position error, and the stiffness normal to the trajectory was decreased to reduce the lateral force.

Fig. 8 shows the learning effects of the two performance indexes Δ_{rms} and $\|\mathbf{F}_{\text{normal}}\|$. For all the three virtual trajectories, the time integrals of the performance indexes considerably decreased after learning. Compared with “understeering,” “oversteering” and “exact-steering” show better performance after learning in terms of position error and lateral force.

B. Point-to-Point Movement in an Unknown Force Field

In this task, the planar robotic manipulator makes a point-to-point movement in an unknown force field. The velocity-dependent force is given as

$$\mathbf{F}_{\text{viscous}} = \begin{bmatrix} -10 & 0 \\ 0 & -10 \end{bmatrix} \begin{Bmatrix} \dot{x} \\ \dot{y} \end{Bmatrix}. \quad (29)$$

The movement of the endpoint of the manipulator is impeded by the force proportional to its velocity, as it moves to the goal position in the force field. This condition is identical to that of the biomechanics study of Kang *et al.* [37]. In their study, the subject moves one’s hand from a point to another point in a velocity-dependent force field, which is same as (29), where the force field was applied by a special apparatus (KINARM, BKIN Technology).

For this task, the performance indexes are chosen as follows:

- 1) the root mean square of the difference between the desired position (virtual trajectory) and the actual position of the endpoint (Δ_{rms});
- 2) the magnitude of the time rate of the torque vector of two arm joints ($\|\dot{\tau}\| = \sqrt{\dot{\tau}_1^2 + \dot{\tau}_2^2}$).

The torque rate represents the power consumption, which can also be interpreted as metabolic costs for human arm movement [38], [39]. By combining the two performance

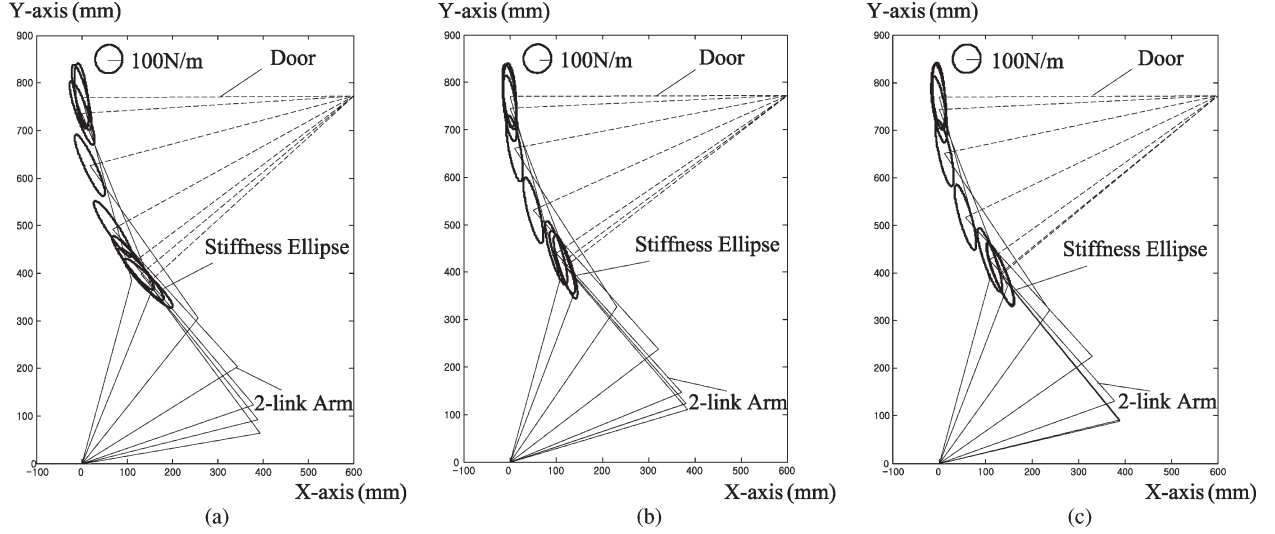


Fig. 7. Stiffness ellipse trajectories. (a) Understeering. (b) Exact Steering. (c) Oversteering.

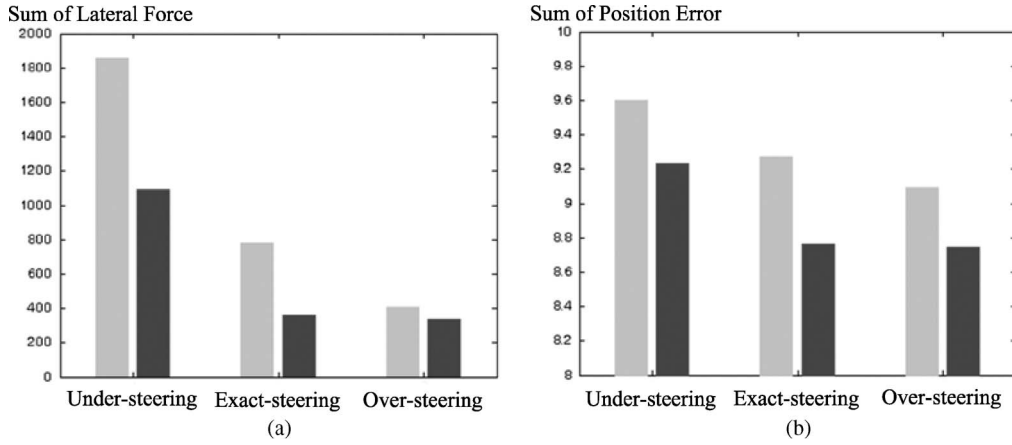


Fig. 8. Learning effects of performance indexes. (a) Position error. (b) Lateral force. (Gray bar: before learning; black bar: after learning.)

indexes, two different rewards are formulated for one episode, as follows:

$$reward1 = \kappa_1 - \sum_{t=1}^N (\Delta_{rms})_t$$

$$reward2 = \nu_1 \left(\kappa_1 - \sum_{t=1}^N (\Delta_{rms})_t \right) + \nu_2 \left(\kappa_2 - \sum_{t=1}^N \|\dot{\tau}\|_t \right)$$

where ν_1 and ν_2 are the weighting factors, and κ_1 and κ_2 are constants. The reward is a weighted linear combination of time integrals of two performance indexes.

The learning parameters were chosen as follows: $\alpha = 0.05$, $\beta = 0.99$, and $\gamma = 0.99$. The change limits for action are set as $[-10, 10]$ degrees for the orientation, $[-2, 2]$ for the major/minor ratio, and $[-200\pi, 200\pi]$ for the area of stiffness ellipse. The initial ellipse before learning was set to be circular with the area of 2500π . The same physical properties as in [37] were chosen for dynamic simulations (Table III).

Fig. 9 shows the change of stiffness ellipse trajectory before and after learning. Before learning, the endpoint of the manipulator was not even able to reach the goal position [Fig. 9(a)].

TABLE III
PHYSICAL PROPERTIES OF A TWO-LINK ARM MODEL

	Length(m)	Mass(Kg)	Inertia(Kg · m ²)
Link 1	0.11	0.20	0.00023
Link 2	0.20	0.18	0.00069

Fig. 9(b) and (c) compares the stiffness ellipse trajectories after learning using two different rewards (*reward1* and *reward2*). As can be seen in figure, for both rewards, the major axis of the stiffness ellipse was directed to the goal position to overcome the resistance of the viscous force field.

Fig. 10 compares the effects of two rewards on the changes of two performance indexes (Δ_{rms} and $\|\dot{\tau}\|$) as learning iterates. While the choice of reward does not affect the time integral of Δ_{rms} , the time integral of $\|\dot{\tau}\|$ was considerably suppressed by using *reward2* in learning.

The results of dynamic simulations are comparable with the biomechanics study of Kang *et al.* [37]. The results of their study suggest that the human actively modulates the major axis toward the direction of the external force against the motion, which is in accordance with our results.

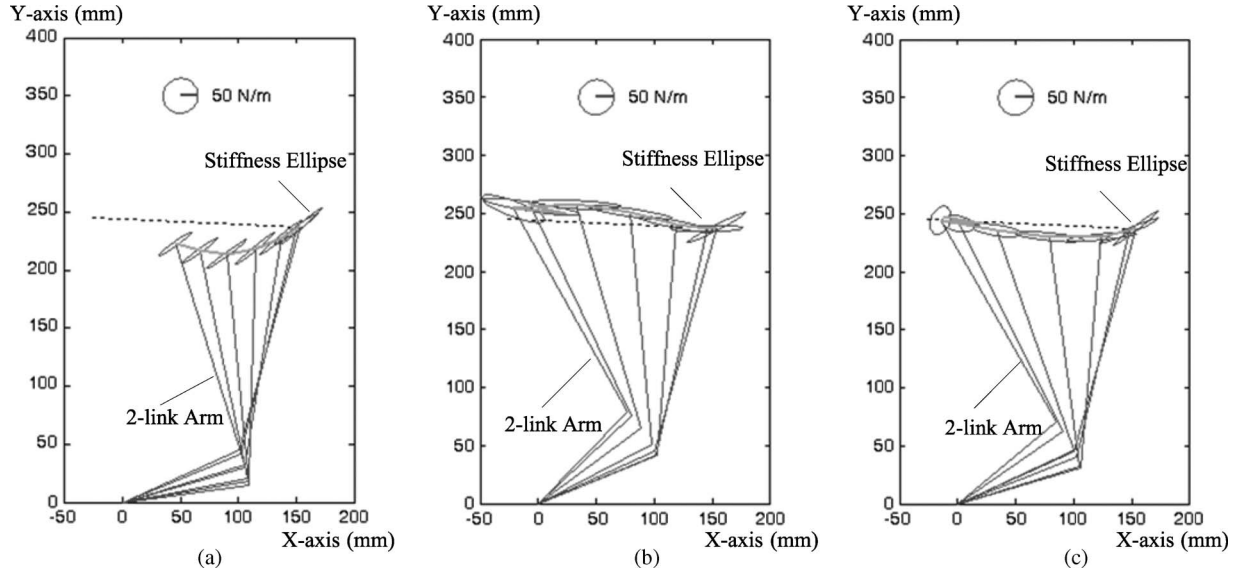


Fig. 9. Stiffness ellipse trajectories. (Dotted line: virtual trajectory; solid line: actual trajectory.) (a) Before learning. (b) After learning (*reward1*). (c) After learning (*reward2*).

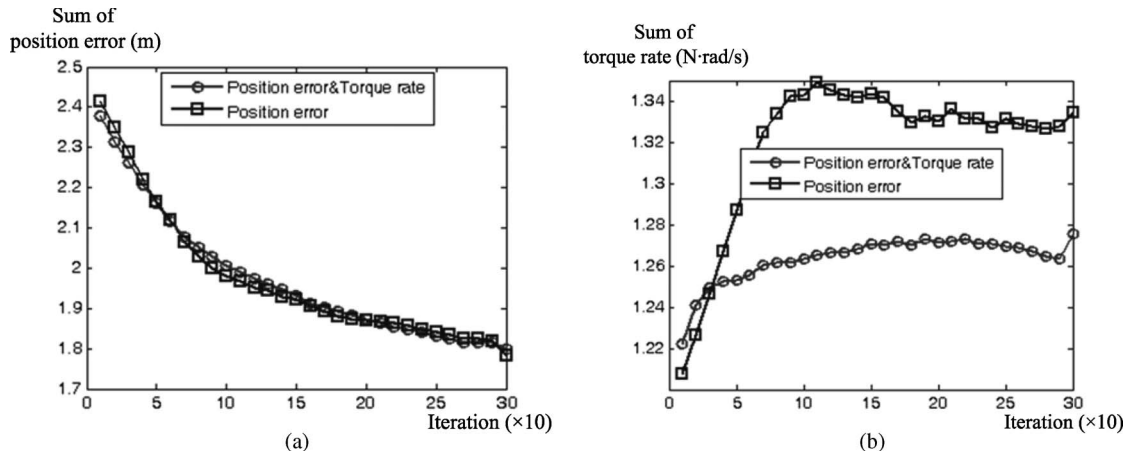


Fig. 10. Learning effects of performance indexes (average of ten learning trials). (a) Position error. (b) Torque rate.

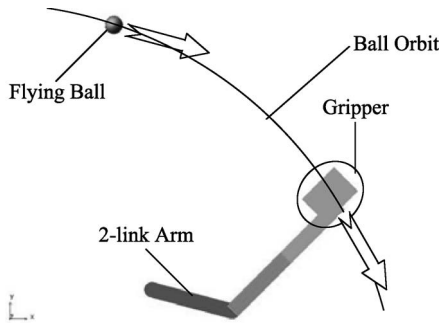


Fig. 11. Catching a flying ball.

C. Catching a Flying Ball

In this task, the planar robotic arm catches a flying ball, as illustrated in Fig. 11. The simulation was performed using the physical properties of the arm, as listed in Table III. The main issues in ball-catching tasks would be how to detect the ball trajectory and how to reduce the impulsive force between the

ball and the end-effector. This paper focuses on the latter and assumes that the ball trajectory is known in advance.

When a human catches a ball, one moves one's arm backward to reduce the impulsive contact force. By considering the human ball catching, the task is modeled as follows: A ball is thrown to the end-effector of the robot arm. The time for the ball to reach the end-effector is approximately 0.8 s. After the ball is thrown, the arm starts to move following the parabolic orbit of the flying ball. While the end-effector is moving, the ball is caught and then moves to the goal position together. The robot is set to catch the ball when the end-effector is moving at its highest speed to reduce the impulsive contact force between the ball and the end-effector. The impulsive force can also be reduced by modulating the stiffness ellipse during the contact.

The learning parameters were chosen as follows: $\alpha = 0.05$, $\beta = 0.99$, and $\gamma = 0.99$. The change limits for action are set as $[-10, 10]$ degrees for the orientation, $[-2, 2]$ for the major/minor ratio, and $[-200\pi, 200\pi]$ for the area of the stiffness ellipse. The initial ellipse before learning was set to be circular with an area of 10000π .

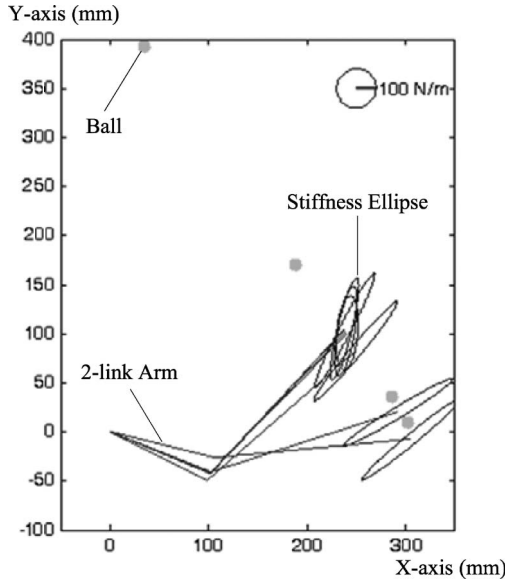


Fig. 12. Stiffness ellipse trajectory during the ball-catching task.

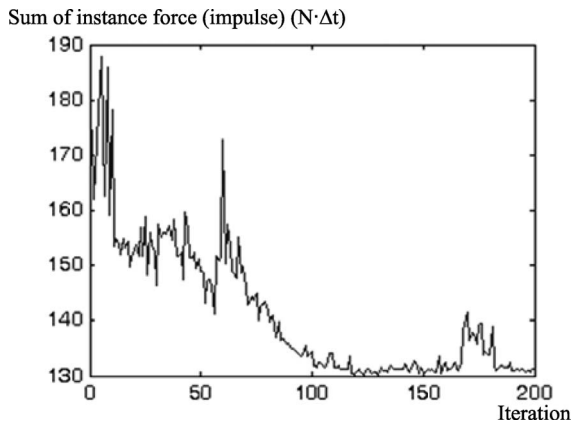


Fig. 13. Learning effect of the impulse (time integral of the contact force).

For this task, the contact force is chosen as the performance index

$$\|\mathbf{F}_{\text{contact}}\| = \sqrt{F_x^2 + F_y^2}. \quad (30)$$

The reward to be maximized is the impulse (the time integral of contact force) during contact, i.e.,

$$\text{reward} = \kappa - \sum_{t=1}^N \|\mathbf{F}_{\text{contact}}\|_t \Delta t \quad (31)$$

where κ is a constant. Fig. 12 illustrates the change of stiffness during contact after learning. As can be seen in the figure, the stiffness is tuned soft in the direction of the ball trajectory, whereas the stiffness normal to the trajectory is much higher. Fig. 13 shows the change of the impulse as learning continues. As can be seen in the figure, the impulse was considerably reduced after learning.

V. CONCLUSION

Safety in robotic contact tasks has become one of the important issues as robots spread their applications to dynamic human-populated environments. The determination of impedance control parameters for a specific contact task would be the key feature in enhancing the robot performance. This paper has proposed a novel motor learning framework for determining the impedance parameters required for various contact tasks. As a learning framework, we employed reinforcement learning to optimize the performance of contact task. We have demonstrated that the proposed framework enhances contact tasks, such as door opening, point-to-point movement, and ball catching. Indeed, a number of studies on animal brains have shown the evidence of reward-based reinforcement learning in the basal ganglia area of the brain related to motor skill learning [40].

In our future works, we will extend our method to teach a service robot that is required to perform more realistic tasks in 3-D space. For translation and rotation in 3-D space, high-dimensional stiffness *ellipsoids* are required. In addition, we are currently investigating a learning method to develop motor schemata that combine the internal models of contact tasks with the actor-critic algorithm developed in this paper. The motor schema, which develops the internal model and the action strategy altogether through learning, is known to be computationally effective and biologically plausible [4].

REFERENCES

- [1] R. A. Schmidt, "A schema theory of discrete motor skill learning," *Psychol. Rev.*, vol. 82, no. 4, pp. 225–260, Jul. 1975.
- [2] R. A. Schmidt, "Motor schema theory after 27 years: Reflections and implications for a new theory," *Res. Quart. Exercise Sport*, vol. 74, no. 4, pp. 366–375, 2003.
- [3] M. Cohen and T. Flash, "Learning impedance parameters for robot control using an associative search network," *IEEE Trans. Robot. Autom.*, vol. 7, no. 3, pp. 382–390, Jun. 1991.
- [4] J. Izawa, T. Kondo, and K. Ito, "Biological robot arm motion through reinforcement learning," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2002, vol. 4, pp. 3398–3403.
- [5] S. Jung, S. B. Yim, and T. C. Hsia, "Experimental studies of neural network impedance force control of robot manipulator," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2001, pp. 3453–3458.
- [6] T. Tsuji, M. Terauchi, and Y. Tanaka, "Online learning of virtual impedance parameters in non-contact impedance control using neural networks," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 34, no. 5, pp. 2112–2118, Oct. 2004.
- [7] T. Flash, "The control of hand equilibrium trajectories in multi-joint arm movement," *Biol. Cybern.*, vol. 57, no. 4/5, pp. 257–274, Nov. 1987.
- [8] R. S. Sutton and A. G. Barto, *Reinforcement Learning: An Introduction*. Cambridge, MA: MIT Press, 1998.
- [9] Y. Engel, S. Mannor, and R. Meir, "Bayes meets Bellman: The Gaussian process approach to temporal difference learning," in *Proc. 20th Int. Conf. Mach. Learn.*, 2003, pp. 154–161.
- [10] D. Liu, H. Javaherian, O. Kovalenko, and T. Huang, "Adaptive critic learning techniques for engine torque and air-fuel ratio control," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 38, no. 4, pp. 988–993, Aug. 2008.
- [11] Q. Yang, J. B. Vance, and S. Jagannathan, "Control of nonaffine nonlinear discrete-time systems using reinforcement-learning-based linearly parameterized neural networks," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 38, no. 4, pp. 994–1001, Aug. 2008.
- [12] S. Ray, G. K. Venayagamoorthy, B. Chaudhuri, and R. Majumder, "Comparison of adaptive critic-based and classical wide-area controllers for power systems," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 38, no. 4, pp. 1002–1007, Aug. 2008.

- [13] C. Lu, J. Si, and X. Xie, "Direct heuristic dynamic programming for damping oscillations in a large power system," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 38, no. 4, pp. 1008–1013, Aug. 2008.
- [14] J. Valasek, J. Doebbler, M. D. Tandale, and A. J. Meade, "Improved adaptive-reinforcement learning control for morphing unmanned air vehicles," *IEEE Trans. Syst., Man, Cybern. B, Cybern.*, vol. 38, no. 4, pp. 1014–1020, Aug. 2008.
- [15] J. Peters, S. Vijayakumar, and S. Schaal, "Reinforcement learning for humanoid robotics," in *Proc. 3rd IEEE-RAS Int. Conf. Humanoid Robots*, 2003, pp. 225–230.
- [16] J. Peters and S. Schaal, "Policy gradient methods for robotics," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2006, pp. 2219–2225.
- [17] J. Peters and S. Schaal, "Natural actor-critic," *Neurocomputing*, vol. 71, no. 7–9, pp. 1180–1190, Mar. 2008.
- [18] J. Park, J. Kim, and D. Kang, "An RLS-based natural actor-critic algorithm for locomotion of a two-linked robot arm," in *Proc. Int. Conf. CIS, Part I*, 2005, vol. 3801, pp. 65–72.
- [19] B. Kim, B. Kang, S. Park, and S. Kang, "Learning robot stiffness for contact tasks using the natural actor-critic," in *Proc. IEEE Int. Conf. Robot. Autom.*, 2008, pp. 3832–3837.
- [20] T. Flash and N. Hogan, "The coordination of arm movements: An experimentally confirmed mathematical model," *J. Neurosci.*, vol. 5, no. 7, pp. 1688–1703, Jul. 1985.
- [21] N. Hogan, "Impedance control: An approach to manipulation: Part I. Theory, Part II. Implementation, Part III. Application," *Trans. ASME, J. Dyn. Syst. Meas. Control*, vol. 107, pp. 12–24, 1985.
- [22] H. Asada and J.-J. E. Slotine, *Robot Analysis and Control*. Hoboken, NJ: Wiley, 1986.
- [23] S. Park and T. B. Sheridan, "Enhanced human-machine interface in braking," *IEEE Trans. Syst., Man, Cybern. A, Syst., Humans*, vol. 34, no. 5, pp. 615–629, Sep. 2004.
- [24] G. Strang, *Linear Algebra and Its Applications*. Orlando, FL: Harcourt Brace Jovanovich, 1988.
- [25] H. Lipkin and T. Patterson, "Generalized center of compliance and stiffness," in *Proc. IEEE Int. Conf. Robot. Autom.*, 1992, pp. 1251–1256.
- [26] H. Kazerooni, P. K. Houpt, and T. B. Sheridan, "The fundamental concepts of robust compliant motion for robot manipulators," in *Proc. IEEE Int. Conf. Robot. Autom.*, 1986, vol. 1, pp. 418–427.
- [27] J. Won, "The control of constrained and partially constrained arm movement," M.S. thesis, Dept. Mech. Eng., MIT, Cambridge, MA, 1993.
- [28] R. S. Sutton, D. McAllester, S. Singh, and Y. Mansour, "Policy gradient methods for reinforcement learning with function approximation," in *Advances in Neural Information Processing Systems*, vol. 12. Cambridge, MA: MIT Press, 2000, pp. 1057–1063.
- [29] W. B. Powell, *Approximate Dynamic Programming: Solving the Curses of Dimensionality*. Hoboken, NJ: Wiley-Interscience, 2007.
- [30] T. K. Moon and W. C. Stirling, *Mathematical Methods and Algorithm for Signal Processing*. Upper Saddle River, NJ: Prentice-Hall, 2000.
- [31] V. Konda and J. N. Tsitsiklis, "Actor-critic algorithms," *SIAM J. Control Optim.*, vol. 42, no. 4, pp. 1143–1166, 2003.
- [32] L. Baird, "Advantage updating," Wright Lab., Wright-Patterson Air Force Base, Dayton, OH, Tech. Rep. WL-TR-93-1146, 1993.
- [33] S. Amari, "Natural gradient works efficiently in learning," *Neural Comput.*, vol. 10, no. 2, pp. 251–276, Feb. 1998.
- [34] J. Boyan, "Least-squares temporal difference learning," in *Proc. Int. Conf. Mach. Learn.*, 1999, pp. 49–56.
- [35] X. Xu, H. He, and D. Hu, "Efficient reinforcement learning using recursive least-squares methods," *J. Artif. Intell. Res.*, vol. 16, pp. 259–292, 2002.
- [36] R. J. Williams, "Simple statistical gradient-following algorithms for connectionist reinforcement learning," *Mach. Learn.*, vol. 8, no. 3/4, pp. 229–256, May 1992.
- [37] B. Kang, B. Kim, S. Park, and H. Kim, "Modeling of artificial neural network for the prediction of the multi-joint stiffness in dynamic condition," in *Proc. IEEE/RSJ Int. Conf. Intell. Robots Syst.*, 2007, pp. 1840–1845.
- [38] D. W. Franklin, U. So, M. Kawato, and T. E. Milner, "Impedance control balances stability with metabolically costly muscle activation," *J. Neurophysiol.*, vol. 92, no. 5, pp. 3097–3105, 2004.
- [39] Y. Uno, R. Suzuki, and M. Kawato, "Formation and control of optimal trajectory in human multi-joint arm movement: Minimum torque change model," *Biol. Cybern.*, vol. 61, no. 2, pp. 89–101, Jun. 1989.
- [40] W. Schultz, P. Dayan, and P. Montague, "A neural substrate of prediction and reward," *Science*, vol. 275, no. 5306, pp. 1593–1599, Mar. 1997.



Byungchan Kim received the B.S. and M.S. degrees from Korea University, Seoul, Korea, in 2005 and 2007, respectively, both in mechanical engineering.

He is currently a Commissioned Research Scientist with the Center for Cognitive Robotics Research, Korea Institute of Science and Technology, Seoul, Korea. His research interests include machine learning, dynamics systems and control, and their applications to robot manipulation.



Jooyoung Park received the B.S. degree in electrical engineering from Seoul National University, Seoul, Korea, in 1983 and the Ph.D. degree in electrical and computer engineering from the University of Texas at Austin, Austin, in 1992.

Since 1993, he has been with Korea University, Jochiwon, Korea, where he is currently a Professor with the Department of Control and Instrumentation Engineering. His recent research interests are in the area of reinforcement learning and kernel methods.



Shinsuk Park received the B.S. and M.S. degrees in mechanical design and production engineering from Seoul National University, Seoul, Korea, in 1989 and 1991, respectively, and the Ph.D. degree in mechanical engineering from the Massachusetts Institute of Technology, Cambridge, in 1999.

From 2000 to 2002, he was a Postdoctoral Research Fellow with the Biorobotics Laboratory, Harvard University, Cambridge. From 2002 to 2004, he was a Visiting Assistant Professor with the Department of Mechanical Engineering, Keio University, Tokyo, Japan. He is currently an Associate Professor with the Department of Mechanical Engineering, Korea University, Seoul. His research interests include dynamics systems and control and their applications to human-machine interface design and biomedical engineering problems.



Sungchul Kang received the B.S., and M.S., and Ph.D. degrees from Seoul National University, Seoul, Korea, in 1989, 1991, and 1998, respectively, all in mechanical design and production engineering.

From 1999 to 2000, he was a Postdoctoral Research Fellow with the Mechanical Engineering Laboratory, National Institute of Advanced Industrial Science and Technology, Tsukuba Science City, Japan. He is currently a Principal Research Scientist with the Center for Cognitive Robotics Research, Korea Institute of Science and Technology, Seoul.

His research interests include intelligent dependable manipulation, field service robot for hazardous task, haptics, tactile interface, control, and its application.